Instrucciones

A continuación, se indican los pasos a seguir para realizar la actividad. Dividiremos la actividad en dos partes, una para la integración continua (CI, continuous integration) y otra para el despliegue continuo (CD, continuous deployment). Los recursos de apoyo se encuentran en Recursos CD y Recursos CI dentro de la carpeta del enunciado.

CI (Continuous Integration)

En este apartado aprenderemos a crear github workflows. Los github workflows son ficheros ".yml" en los que indicamos pasos (task) que se ejecutarán según el trigger que elijamos. Un trigger simplemente es un evento dentro de nuestro repositorio, como por ejemplo el inicio de un pull request.

- Preparar el repositorio:
    - Debemos crear un repositorio nuestro vacío en donde meteremos el código que os proporciono.
    - El código a copiar se encuentra en el siguiente repo: https://github.com/joseagcv/pontia-mlops-tutorial.

      Sugiero clonarlo en local, copiar todos los archivos **EXCEPTO** el ".git" a nuestro repositorio vacío y subirlo. Podéis también hacer un fork, pero luego tendréis que dejar de trackear algunas carpetas borrandolas del caché.
    - Hay que agregar al .gitignore las carpetas "./model" y "./data", ya que lo normal es que no se suban ni datos ni modelos entrenados al repositorio. Si creáis un entorno virtual de python, es necesario ignorarlo también.
    - Copiar el requirements.txt en nuestro repositorio.
    - Configurar el código para que utilice variables de entorno, en específico **MLFLOW_URL**.
    - Arrancar un servidor local de mlflow con el siguiente comando:
      docker run -d --name mlflow -p 5000:5000 ghcr.io/mlflow/mlflow:v3.7.0 mlflow server --host 0.0.0.0 --port 5000
    - Probar el código en local.
    - Una vez probado, subir a la rama main.

- Agregamos el workflow de build. Queremos que nuestro código principal sea el que genere los artefactos. Para esto, es necesario crear un workflow que haga esto por nosotros.
    - Debemos crear una carpeta en la raiz de nuestro repositorio con el siguiente nombre exacto ".github/workflows". En esta carpeta meteremos todos nuestros workflows.

    - Copiamos el build.yml a esta carpeta.

    - Cambiamos la versión de python a la "3.12"

    - Reparamos el pip install.

    - Descargamos los datos con un task dentro del pipeline. Se deja código de EJEMPLO de cómo realizar una descarga en download-data.yml. Es solo un ejemplo, deberéis copiar solo lo necesario en la build.yml.

    - Publicar el resultado en artifactory de github utilizando un "action/upload-artifact@v4"

    - Crear variables de entorno en Github Environment. Como nombre de Environment, debemos poner "Staging". Se deben crear secrets:
        - AZURE_TENANT_ID:
        - AZURE_CLIENT_ID:
        - AZURE_CLIENT_SECRET:

      Y variables de entorno:
        - MLFLOW_URL:
        - EXPERIMENT_NAME: vuestronombre-adult-income

- Agregar el workflow de integration.
    - Copiamos el integration.yml a esta carpeta.

    - Cambiamos la versión de python a la "3.12"

    - Los tests se ejecutarán pero terminarán fallando ya que no existen los modelos entrenados, por lo que no se generará reporte y el último paso fallará. Es necesario agregar un "if: always()" al último paso.

    - Realizar un pull request y verificar que se ejecuta el workflow.

- Agregar normas al repositorio. NOTA: Estas restricciones sólo se aplicarán si el repositorio es público ya que para que se ejecuten en uno privado habría que pagar.
    - Agregamos un Ruleset sobre la rama main:
        - "Restrict deletions"
        - "Block force pushed"
        - "Require status checks to pass". Sobre nuestra pipeline.
        - "Require branches to be up to date before merging"

    - Probar ahora quitando el if:always() y verificar que no es posible realizar el merge.

    - Colocar de nuevo el if:always() de nuevo y verificar que funciona.

    - Descargamos los datos para realizar los tests (igual que en el paso anterior en el entrenamiento).
    - Descargamos el modelo usando el download-model.py.
    - Quitar el if:always() ya que los tests deben pasar.

CD (continuous delivery)

En la carpeta de Recursos CD se encuentra una pequeña app de fastapi la cual queremos construir su imagen docker y luego desplegarla.

Vamos a construir la imagen, la queremos publicar en nuestro repositorio privado de docker hub. Esto lo haremos con el workflow deploy. Las instrucciones son:

- Copiar la carpeta deployment en nuestro repositorio.
- Limpiar un poco la imagen de docker.
- Crear un repo privado en docker hub. Crear credenciales con permisos para escribir en este repo.
- Crear las siguientes variables y secrets:
    - Secret:
        - DOCKERHUB_TOKEN:
        - DOCKERHUB_USERNAME:
        - AZURE_SUBSCRIPTION_ID:
    - variables:
        - DOCKERHUB_REPO:
        - CONTAINER_APP:
        - AZURE_RESOURCE_GROUP:
        - IMAGE_NAME:
- Crear un pull request e integrar este pipeline al código. Se debería de ejecutar todo si está configurado correctamente el pipeline.